CASE STUDY 02 · IN ACTIVE DEVELOPMENT

Vigil
Fall detection & live location

A free safety app for iPhone and Android. A small neural network on the phone watches for falls; if you don't answer "Are you okay?", the people in your circle get a full-screen alert with your live location.

ROLE	PLATFORMS	ML
Sole developer: app, ML model, backend	Android & iOS (Flutter)	1D CNN · SisFall · TensorFlow Lite

BACKEND	STATUS
Firebase Auth · Firestore · Hosting	Working app · model fine-tuning

01 / the problem

The problem

When someone falls and can't get up — an elderly parent, a person working alone, anyone — the minutes before help arrives matter. Most fall-detection solutions rely on a dedicated watch or a paid service. I wanted something that works with **only the phone people already carry**, costs nothing, and alerts the specific people who would actually come.

02 / what it does

What it does

On-device fall detection

A neural network runs on the phone itself — no audio or motion data leaves the device for detection.

"Are you okay?" countdown

After a suspected fall, a countdown gives the person a chance to cancel a false alarm.

Full-screen circle alerts

If there's no answer, everyone in the circle gets a full-screen alert with a loud alarm and strong vibration.

SOS button

Trigger the same alert manually at any time.

Live location & ETA

Helpers tap "on my way"; the person in trouble and the circle see them approach on a map with an ETA.

Invite-link circles

Start alone, then add family by sharing a link. Circles work like groups, with admins and permissions.

Privacy controls

Per-person choices about who sees location and who receives alerts.

Movement recorder

A built-in recorder to capture real phone movement for fine-tuning the model.

03 / the ml model

The ML model

SisFall dataset

Accelerometer recordings of falls and daily activities

→

Preprocess

Resampled to 50 Hz, cut into windows

→

1D CNN

Trained in Google Colab

→

Evaluate

Subject-independent split

TensorFlow Lite
Shipped inside the app

The model is evaluated **subject-independently**: people in the test set are never seen in training, which is closer to how it behaves for a new user.

In the app, inference is wrapped in simple logic that keeps the battery and false alarms down: the model only runs after an **impact above 1.6 g**, checks every **0.5 s**, and has a **60-second cooldown** after an alert.

HONEST STATUS

Lab datasets are recorded with sensors strapped to the body, but a phone sits in a pocket, a hand or a bag. In real use the first model raised false alarms during normal movement, so I built the recorder and a fine-tuning notebook that mixes SisFall with my own phone recordings at a low learning rate. Final evaluation numbers will be published here once the real-device data is complete.

04 / architecture

Architecture

Flutter app
Sensors, model, alerts, maps

Firebase Auth
Google, email or anonymous sign-in

Cloud Firestore
Circles, members, alerts, locations — guarded by security rules

Firebase Hosting
Invite links: `/join/CODE` opens the app

Invite links are real web URLs. On Android they open the app directly through verified app links; on iPhone (without a paid Apple account) a small page offers an "Open in Vigil" button that hands the code to the app.

05 / challenges

Challenges

False alarms from everyday movement

Sitting down hard or dropping the phone can look like a fall. The answer was better data from real phone placements (pocket, hand, bag, table), a fine-tuning pipeline, and the countdown so a person can cancel.

An alarm that must be heard

An alert is useless if it follows the media volume. I configured the audio session so the alarm stays loud with strong vibration even when the phone volume is turned down.

Alerts while the app is in the background

iOS limits background work. Getting alerts to arrive reliably when the app isn't open required notification and background handling specific to each platform.

Maps for everyone

The default map tiles repeated the world and showed local-language labels. I switched to tiles with English labels, limited zoom to one world, and added a satellite toggle.

06 / what i learned

What I learned

- The gap between a model that scores well on a dataset and one that works in a pocket — and how to close it with data.
- Building a real-time multi-user app on Firestore with security rules that protect location data.
- Mobile platform realities: background limits, deep links, audio sessions, permissions.

07 / documentation

Documentation

Project layout

`app/` is the Flutter app only. `ml/` holds the Colab notebooks (`vigil_fall_detection.ipynb`, `vigil_finetune.ipynb`) and the exported `vigil_fall_model.tflite` with its config. `web/` is the Firebase Hosting site for invite links. `docs/` holds the architecture notes and security rules.

Run the app

1. Install Flutter and the FlutterFire CLI.
2. `flutterfire configure` with your own Firebase project (config files are not committed).
3. `flutter run` on an Android emulator or a connected iPhone.

Retrain the model

Open `vigil_finetune.ipynb` in Colab with a T4 GPU, point it at your recordings, run all cells, and copy the new `.tflite` and config into `app/assets/models/`. No code changes are needed.